3030AFE Take-Home Exam (Q-only)

`Read the Exam Guide carefully -- penality applies if you don't follow the Guide.`

Exam Guide

- Duration: 4 hours, **10am to 2pm, 11/10/2024 (Brisbane time)**.
- Answer ALL Three (3) questions.
- Total marks: 50.
- Data for this assignment are available from the course L@G website.
- Use Jupyter Notebook/Python to achieve the answers.
- Type up your comments/discussions with Markdown.
- Submit two files: pdf & ipynb (as Assignments 1 & 2). The files should include your name and student ID, Python codes and outcomes, and comments/discussions/explanations.
- Use your `student ID` without s to set up random_state (for replication).
- Use the default setting for modelling (except for random_state) unless the values of relevant hyperparameters are clearly specified.
- Use codes learned from the course -- justification is required if codes beyond those taught in the course are used.
- This is an individual assessment item -- if found to have cheated, all submissions involved would receive a mark of zero for this assessment item.

Liam Barry

s5221080

Libraries

In [1]:
```
# essential libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
plt.style.use('seaborn-v0_8-white')
sns.set_style('darkgrid')

# statsmodels
import statsmodels.api as sm
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.holtwinters import SimpleExpSmoothing, ExponentialSmoothing

# sklearn
from sklearn.model_selection import train_test_split, KFold, cross_val_score, StratifiedKFold, GridSearchCV
from sklearn import metrics
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import confusion_matrix, roc_curve, roc_auc_score, precision_score, recall_score, f1_score, classification_report
from sklearn.linear_model import LinearRegression, Lasso, LassoCV, LogisticRegression
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor, GradientBoostingClassifier, GradientBoostingRegressor
from sklearn.neural_network import MLPClassifier, MLPRegressor
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.inspection import permutation_importance
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import CountVectorizer
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier

# skforecast
import skforecast
from skforecast.ForecasterAutoreg import ForecasterAutoreg
from typing_extensions import Self

# nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer as SIA

# shap
import shap

import warnings
warnings.filterwarnings("ignore")
```

Question 1 (15 marks)

Use the data in `House.csv` to answer this question (load it as a dataframe called `HS`). The file contains information on 1800 houses: `SalePrice` (price of a house in thousands of dollars), `LivArea` (living area of the house in square feet), `Quality` (a score=1,2,3 indicating the quality of the house).

1. Fit a linear regression model to `SalePrice` on the 2 features with the entire dataset. Report the regression result and discuss the effect and significance of each feature contributing to fitting the target variable (`SalePrice`).
2. Covert `Quality` into dummies (binary variables) and re-run the regression as Part 1 by replacing `Quality` by its corresponding dummies. Does the revised model perform better? Explain.
3. Based on the fitted model of Part 2, make a prediction of SalePrice for the following two houses: House_1 (`LivArea = 1000`, `Quality=3`) and House_2 (`LivArea = 2000`, `Quality=1`). Compare the predictions.
4. Redo the model with dummy variables with a decision tree (with `max_depth=5`). Make a prediction of `SalePrice` for the following two houses: House_1 (`LivArea = 1000`, `Quality=3`) and House_2 (`LivArea = 2000`, `Quality=1`) with the Decision Tree.
5. Provide a `compelling comparison` between the Regression Model from Part 2 and the Decision Tree model from Part 4 in terms of prediction power. Based on the model results, explain which model you would rely on for predicting the sale prices of `House_1` and `House_2`, and justify your choice.

In [2]:
```
# load data
HS = pd.read_csv('House.csv')
HS.head()
```

Out[2]:

	Quality	LivArea	SalePrice
0	2	1710	208.5
1	2	1262	181.5
2	2	1786	223.5
3	2	1717	140.0
4	3	2198	250.0

1. Fit a linear regression model to SalePrice on the 2 features with the entire dataset. Report the regression result and discuss the effect and significance of each feature contributing to fitting the target variable (SalePrice).

In [3]:
```
X = HS.drop('SalePrice', axis = 1)
y = HS['SalePrice']
```

In [4]:
```
LR = LinearRegression()
LR.fit(X, y)

# intercept & slope
print("intercept =", LR.intercept_)
print("slopes (weight) =", LR.coef_)
```

```
intercept = -82.88858520537514
slopes (weight) = [70.1877725   0.07909453]
```

In [5]:
```
# prediction
yp_LR = LR.predict(X)
# MSE (mean squared error)
print("MSE =", mean_squared_error(y, yp_LR))
```

```
MSE = 1931.1328646284544
```

In [6]:
```
XX = sm.add_constant(X)
results = sm.OLS(y, XX).fit()
# Print the summary, which includes coefficients, p-values, and other statistics
print(results.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:              SalePrice   R-squared:                       0.676
Model:                            OLS   Adj. R-squared:                  0.676
Method:                 Least Squares   F-statistic:                     1875.
Date:                Fri, 11 Oct 2024   Prob (F-statistic):               0.00
Time:                        13:31:47   Log-Likelihood:                -9363.4
No. Observations:                1800   AIC:                         1.873e+04
Df Residuals:                    1797   BIC:                         1.875e+04
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const        -82.8886      4.584    -18.084      0.000     -91.878     -73.899
Quality       70.1878      2.339     30.002      0.000      65.600      74.776
LivArea        0.0791      0.002     33.012      0.000       0.074       0.084
==============================================================================
Omnibus:                      360.342   Durbin-Watson:                   1.898
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1836.663
Skew:                           0.848   Prob(JB):                         0.00
Kurtosis:                       7.649   Cond. No.                     7.46e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 7.46e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Both Quality and LivArea significantly contribute to predicting SalePrice, with p-values less than 0.05. The coefficient for Quality (70.19) indicates that increasing the quality rating by one level increases SalePrice by 70.19. The coefficient for LivArea (0.0791) means that increasing the living area by one square foot increases SalePrice by 0.0791. The R-squared value of 0.676 suggests that 67.6% of the variance in SalePrice is explained by these features, indicating a good model fit. The mean squared error (MSE) of 1931.13 represents the average squared difference between actual and predicted values, giving an estimate of the prediction error.

2. Convert Quality into dummies (binary variables) and re-run the regression as Part 1 by replacing Quality by its corresponding dummies. Does the revised model perform better? Explain.

In [7]:
```
# using get_dummies for transformation: turn categorical variables --> binary/dummy variables # using drop_frist to avoid multicollinearity
HS = pd.get_dummies(HS, columns=['Quality'], drop_first=True, dtype=int)
HS.head(10)
```

Out[7]:

	LivArea	SalePrice	Quality_2	Quality_3
0	1710	208.5	1	0
1	1262	181.5	1	0
2	1786	223.5	1	0
3	1717	140.0	1	0
4	2198	250.0	0	1
5	1362	143.0	1	0
6	1694	307.0	0	1
7	2090	200.0	1	0
8	1774	129.9	1	0
9	1077	118.0	1	0

In [8]:
```
X_revised = HS.drop('SalePrice', axis = 1)
y_revised = HS['SalePrice']
```

In [9]:
```
LR_revised = LinearRegression()
LR_revised.fit(X_revised, y_revised)

# intercept & slope
print("intercept =", LR_revised.intercept_)
print("slopes (weight) =", LR_revised.coef_)
```

```
intercept = 18.636814063276717
slopes (weight) = [7.61118928e-02 3.59284285e+01 1.31807805e+02]
```

In [10]:
```
# prediction
yp_LR_revised = LR_revised.predict(X_revised)
# MSE (mean squared error)
print("MSE =", mean_squared_error(y_revised, yp_LR_revised))
```

```
MSE = 1769.6648944094866
```

In [11]:
```
XX_revised = sm.add_constant(X_revised)
results_revised = sm.OLS(y_revised, XX_revised).fit()
# Print the summary, which includes coefficients, p-values, and other statistics
print(results_revised.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:              SalePrice   R-squared:                       0.703
Model:                            OLS   Adj. R-squared:                  0.703
Method:                 Least Squares   F-statistic:                     1418.
Date:                Fri, 11 Oct 2024   Prob (F-statistic):               0.00
Time:                        13:31:47   Log-Likelihood:                -9284.8
No. Observations:                1800   AIC:                         1.858e+04
Df Residuals:                    1796   BIC:                         1.860e+04
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         18.6368      4.101      4.544      0.000      10.593      26.680
LivArea        0.0761      0.002     33.005      0.000       0.072       0.081
Quality_2     35.9284      3.490     10.295      0.000      29.083      42.773
Quality_3    131.8078      4.530     29.097      0.000     122.923     140.692
==============================================================================
Omnibus:                      297.985   Durbin-Watson:                   1.908
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1659.310
Skew:                           0.654   Prob(JB):                         0.00
Kurtosis:                       7.518   Cond. No.                     9.25e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 9.25e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

In the revised model, Quality was converted into dummy variables (Quality_2 and Quality_3), and the model shows an improved fit compared to the initial regression. The R-squared value increased from 0.676 to 0.703, meaning 70.3% of the variance in SalePrice is now explained by the features, indicating a better model fit. The mean squared error (MSE) decreased from 1931.13 to 1769.66, suggesting improved prediction accuracy. The dummy variables for Quality have positive and significant coefficients, indicating that higher quality scores positively affect SalePrice. Overall, converting Quality into dummy variables has improved the model's performance.

3. Based on the fitted model of Part 2, make a prediction of SalePrice for the following two houses: House_1 (LivArea = 1000, Quality=3) and House_2 (LivArea = 2000, Quality=1). Compare the predictions.

In [12]:
```
House_1 = pd.DataFrame([[1000, 0, 1]], columns=['LivArea', 'Quality_2', 'Quality_3'])
print(f"SalePrice prediction House_1: {LR_revised.predict(House_1)}")
```

```
SalePrice prediction House_1: [226.55651172]
```

In [13]:
```
House_2 = pd.DataFrame([[2000, 0, 0]], columns=['LivArea', 'Quality_2', 'Quality_3'])
print(f"SalePrice prediction House_2: {LR_revised.predict(House_2)}")
```

```
SalePrice prediction House_2: [170.86059966]
```

In [14]:
```
F_list=tuple(X_revised.columns)

def Prediction():
    X=[]
    for i in F_list:
        print(i, end = " ")
        X.append(int(input()))
    XX=pd.DataFrame([X], columns=(F_list))
    print("Prediction", LR_revised.predict(XX).round(3))
```

In [15]:
```
House1 = Prediction()
```

```
LivArea 1000
Quality_2 0
Quality_3 1
Prediction [226.557]
```

In [16]:
```
House2 = Prediction()
```

```
LivArea 2000
Quality_2 0
Quality_3 0
Prediction [170.861]
```

The predicted SalePrice for House 1 (LivArea = 1000, Quality = 3) is 226.56, while the predicted SalePrice for House 2 (LivArea = 2000, Quality = 1) is 170.86. Despite House 2 having a larger living area, its lower Quality rating leads to a lower predicted price compared to House 1. This indicates that Quality has a significant positive effect on SalePrice, even surpassing the effect of increased LivArea in this case.

4. Redo the model with dummy variables with a decision tree (with max_depth=5). Make a prediction of SalePrice for the following two houses: House_1 (LivArea = 1000, Quality=3) and House_2 (LivArea = 2000, Quality=1) with the Decision Tree.

In [17]:
```
DT = DecisionTreeRegressor(random_state=1080, max_depth=5)
DT.fit(X_revised, y_revised)
```

Out[17]:
```
▾ DecisionTreeRegressor
DecisionTreeRegressor(max_depth=5, random_state=1080)
```

In [18]:
```
House_1 = pd.DataFrame([[1000, 0, 1]], columns=['LivArea', 'Quality_2', 'Quality_3'])
print(f"SalePrice prediction House_1: {DT.predict(House_1)}")
```

```
SalePrice prediction House_1: [183.3625]
```

In [19]:
```
House_2 = pd.DataFrame([[2000, 0, 0]], columns=['LivArea', 'Quality_2', 'Quality_3'])
print(f"SalePrice prediction House_2: {DT.predict(House_2)}")
```

```
SalePrice prediction House_2: [133.68]
```

In [20]:
```
F_list=tuple(X_revised.columns)

def Prediction():
    X=[]
    for i in F_list:
        print(i, end = " ")
        X.append(int(input()))
    XX=pd.DataFrame([X], columns=(F_list))
    print("Prediction", DT.predict(XX).round(3))
```

In [21]:
```
House1 = Prediction()
```

```
LivArea 1000
Quality_2 0
Quality_3 1
Prediction [183.362]
```

In [22]:
```
House2 = Prediction()
```

```
LivArea 2000
Quality_2 0
Quality_3 0
Prediction [133.68]
```

5. Provide a compelling comparison between the Regression Model from Part 2 and the Decision Tree model from Part 4 in terms of prediction power. Based on the model results, explain which model you would rely on for predicting the sale prices of House_1 and House_2, and justify your choice.

In [23]:
```
yp_LR = LR_revised.predict(X_revised)
mse_LR = mean_squared_error(y_revised, yp_LR)
r2_LR = r2_score(y_revised, yp_LR)

yp_DT = DT.predict(X_revised)
mse_DT = mean_squared_error(y_revised, yp_DT)
r2_DT = r2_score(y_revised, yp_DT)

print("Regression Model Performance:")
print(f"Mean Squared Error (MSE): {mse_LR}")
print(f"R-squared (R2): {r2_LR}\n")

print("Decision Tree Model Performance:")
print(f"Mean Squared Error (MSE): {mse_DT}")
print(f"R-squared (R2): {r2_DT}")
```

```
Regression Model Performance:
Mean Squared Error (MSE): 1769.6648944094866
R-squared (R2): 0.7030750804201765

Decision Tree Model Performance:
Mean Squared Error (MSE): 1540.1604428325563
R-squared (R2): 0.7415827046845042
```

The regression model predicts a SalePrice of 226.56 for House 1 and 170.86 for House 2, while the decision tree model predicts 183.36 and 133.68, respectively. The regression model better captures the influence of quality and living area, as the higher quality rating for House 1 results in a higher predicted price, despite a smaller living area. Performance metrics show that the regression model has an MSE of 1769.66 and an R-squared value of 0.703, while the decision tree model has a lower MSE of 1540.16 and a higher R-squared value of 0.742. Although the decision tree slightly outperforms in terms of metrics, the regression model's predictions align better with the expected relationships between quality, living area, and price. Therefore, I would rely on the regression model for predicting SalePrice, as it offers more intuitive results.

Question 2 (17.5 marks)

Use the dataset `Rotten.csv` for answering this question (load it as a dataframe called `Rot`). The dataset contains three columns: `id`, `recommended` (1: recommended, 0: not recommended) and `review` (movie review in text). In total, 900 movie reviews are recorded.

1. Obtain the sentiment scores (`compound, pos, neg, neu`) for each movie review in Rotten. Create a barplot that compares mean values of compound, pos, neg, and neu, respectively color-coded by "recommended" (i.e., use distinct colors for recommended or not). What have you found in the plots? Discuss.
2. Locate the review with the highest compound score and the review with the lowest compound score. Compare the `pos`, `neg`, and `neu` scores of these two reviews. Are they recommended?

Split the data into `85%` train set and `15%` test set randomly. Use the train set for model fitting and the test set for model evaluation in the following parts of the question.

3. Fit a random forest model on the train set with `50` trees using "recommended" as the target variable and `pos` and `neg` as the features. Apply the fitted model to make prediction on the test set. Report the recall and precision scores and evaluate the importance of each feature. Comment on the results.
4. Redo Part 3 with a neural network model. Set one hidden layer with `30` neurons and use `logistic` as the activation function for the model. Report the recall and precision scores and obtain the `SHAP` summary plot (set the plot_type as violin) on the prediction. Comment on the results.
5. Use the results of Parts 3 and 4 to construct `ROC` curves. Does neural network outperform random forest? Explain.

In [24]:
```
# load data
Rot = pd.read_csv('Rotten.csv')
Rot.head()
```

Out[24]:

	id	recommended	review
0	8475_3	0	(NOTE: I thought I'd be the only one writing w...
1	704_10	1	This film, as low budget as it may be, is one ...
2	4016_4	0	There is a DVD published in the UK in 2002 Cod...
3	1886_10	1	I have loved this movie ever since it's debut ...
4	8336_1	0	How the hell did they get this made?! Presenti...

1. Obtain the sentiment scores (compound, pos, neg, neu) for each movie review in Rotten. Create a barplot that compares mean values of compound, pos, neg, and neu, respectively color-coded by "recommended" (i.e., use distinct colors for recommended or not). What have you found in the plots? Discuss.

In [25]:
```
# Assign SIA as sia
# record results in results, first set blank
sia = SIA()
results = []
for row in Rot['review']:
    pol_score = sia.polarity_scores(row)
    pol_score['review'] = row
    results.append(pol_score)
sentimentScores = pd.DataFrame(results)
sentimentScores.head()
```

Out[25]:

	neg	neu	pos	compound	review
0	0.117	0.811	0.072	-0.8570	(NOTE: I thought I'd be the only one writing w...
1	0.042	0.771	0.187	0.9685	This film, as low budget as it may be, is one ...
2	0.089	0.837	0.074	0.0772	There is a DVD published in the UK in 2002 Cod...
3	0.030	0.828	0.141	0.9436	I have loved this movie ever since it's debut ...
4	0.070	0.830	0.099	0.5925	How the hell did they get this made?! Presenti...

In [26]:
```
sentimentScores['recommended'] = Rot['recommended']
sentimentScores.head()
```

Out[26]:

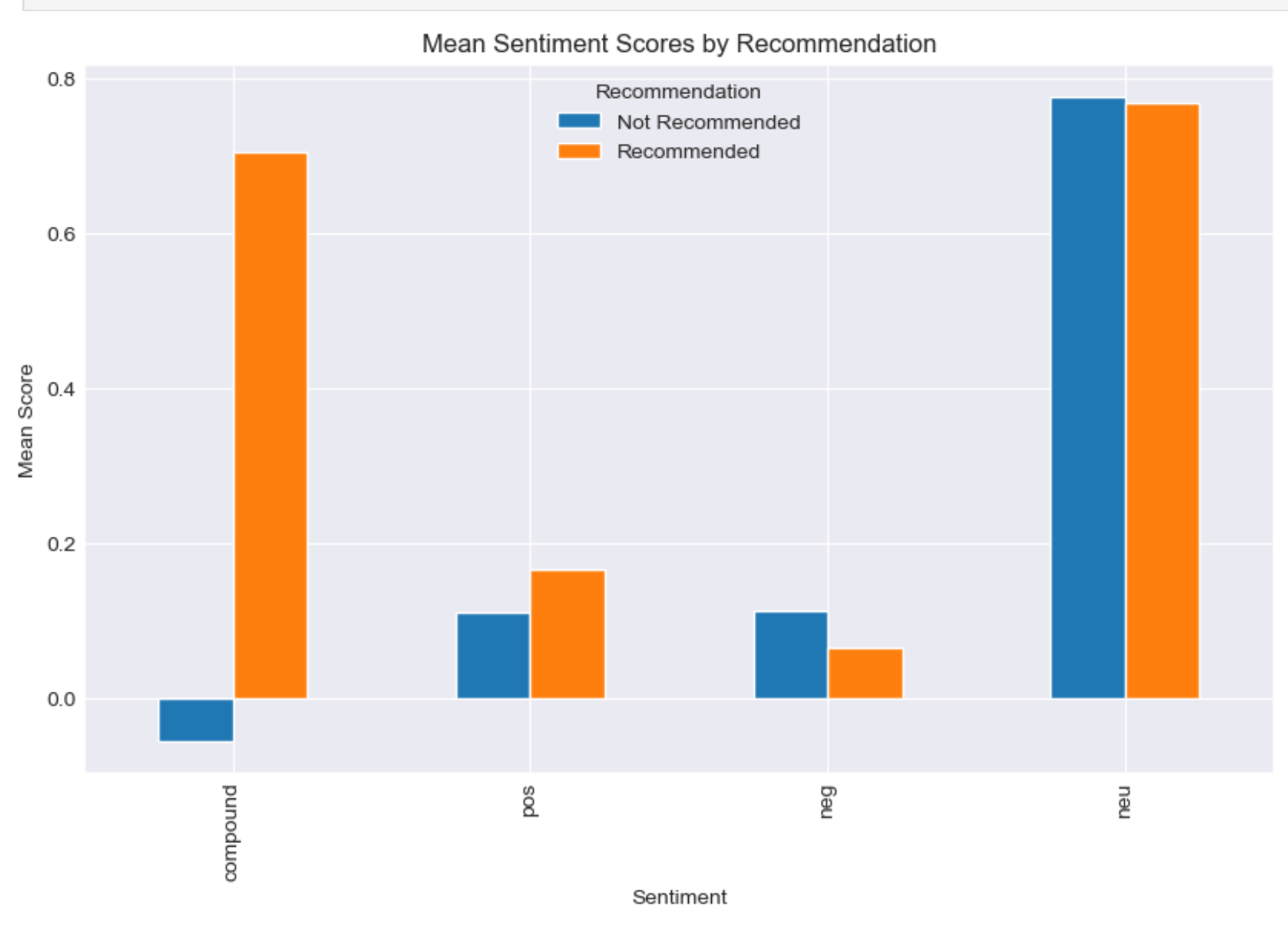
	neg	neu	pos	compound	review	recommended
0	0.117	0.811	0.072	-0.8570	(NOTE: I thought I'd be the only one writing w...	0
1	0.042	0.771	0.187	0.9685	This film, as low budget as it may be, is one ...	1
2	0.089	0.837	0.074	0.0772	There is a DVD published in the UK in 2002 Cod...	0
3	0.030	0.828	0.141	0.9436	I have loved this movie ever since it's debut ...	1
4	0.070	0.830	0.099	0.5925	How the hell did they get this made?! Presenti...	0

In [27]:
```
sentimentScoresMeans = sentimentScores.groupby('recommended')[['compound', 'pos', 'neg', 'neu']].mean()
sentimentScoresMeans
```

Out[27]:

	compound	pos	neg	neu
recommended				
0	-0.054376	0.110628	0.113261	0.776103
1	0.703939	0.167076	0.065019	0.767882

In [28]:
```
sentimentScoresMeans_T = sentimentScoresMeans.T
sentimentScoresMeans_T.columns = ['Not Recommended', 'Recommended']

sentimentScoresMeans_T.plot(kind='bar', figsize=(10, 6))
plt.title('Mean Sentiment Scores by Recommendation')
plt.xlabel('Sentiment')
plt.ylabel('Mean Score')
plt.legend(title='Recommendation')
plt.show()
```



From the bar plot comparing mean sentiment scores for recommended and not recommended movies, it is evident that the "Recommended" category shows a significantly higher compound sentiment score, suggesting more positive overall sentiment in reviews for recommended movies. The "pos" (positive) sentiment is also higher for recommended movies, reinforcing this positive trend. In contrast, the "neg" (negative) sentiment is slightly higher for non-recommended movies, indicating that non-recommended reviews contain more negative sentiment. Interestingly, the "neu" (neutral) scores are almost identical for both categories, which suggests that neutral language is used in reviews regardless of recommendation status. This overall pattern highlights that positive sentiments are strongly associated with recommended movies, while non-recommended ones carry more negative sentiment.

2. Locate the review with the highest compound score and the review with the lowest compound score. Compare the pos, neg, and neu scores of these two reviews. Are they recommended?

In [29]:
```
compMax = sentimentScores['compound'].idxmax()
compMin = sentimentScores['compound'].idxmin()

maxReview = sentimentScores.iloc[compMax]
minReview = sentimentScores.iloc[compMin]
print("Review with Highest Compound Score:")
print(maxReview)
print("\nReview with Lowest Compound Score:")
print(minReview)
```

```
Review with Highest Compound Score:
neg                                                        0.046
neu                                                        0.734
pos                                                         0.22
compound                                                  0.9995
review         I have never seen a show as good as Full House...
recommended                                                    1
Name: 140, dtype: object

Review with Lowest Compound Score:
neg                                                        0.183
neu                                                        0.729
pos                                                        0.088
compound                                                 -0.9987
review         here was no effort put into Valentine to preve...
recommended                                                    0
Name: 264, dtype: object
```

In [30]:
```
print("Recommendation Score of Highest Compound Review:", maxReview['recommended'])
print("Recommendation Score of Lowest Compound Review:", minReview['recommended'])
```

```
Recommendation Score of Highest Compound Review: 1
Recommendation Score of Lowest Compound Review: 0
```

The review with the highest compound score (0.9995) is recommended and has a higher positive sentiment score (0.22) compared to the review with the lowest compound score (-0.9987), which is not recommended and has a lower positive sentiment score (0.088). In contrast, the negative sentiment score is much higher for the lowest compound score review (0.183) than for the highest compound score review (0.046), indicating more negative sentiment in the non-recommended review. Both reviews have nearly identical neutral sentiment scores (0.734 vs. 0.729), suggesting a similar proportion of neutral content in each review.

Split the data into 85% train set and 15% test set randomly. Use the train set for model fitting and the test set for model evaluation in the following parts of the question.

In [31]:
```
X = sentimentScores[['pos', 'neg']]
y = sentimentScores['recommended']
```

In [32]:
```
# Split train & test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15, random_state=1080, shuffle=True)
```

3. Fit a random forest model on the train set with 50 trees using "recommended" as the target variable and pos and neg as the features. Apply the fitted model to make prediction on the test set. Report the recall and precision scores and evaluate the importance of each feature. Comment on the results.

In [33]:
```
# RandomForest
RF = RandomForestClassifier(n_estimators=50, random_state=1080)
RF.fit(X_train, y_train)
# get predicted values
yp_RF = RF.predict(X_test)
```

In [34]:
```
# Calculation based on yp
print("Precision =", precision_score(y_test, yp_RF).round(3))
print("Recall =", recall_score(y_test, yp_RF).round(3))
```

```
Precision = 0.783
Recall = 0.627
```

In [35]:
```
importances = RF.feature_importances_
importances
```

Out[35]:
```
array([0.4913956, 0.5086044])
```

In [36]:
```
FI = pd.DataFrame(importances, index=X_train.columns).T
FI
```

Out[36]:

	pos	neg
0	0.491396	0.508604

The model's performance shows a precision of 0.783 and a recall of 0.627, indicating that while the model is fairly good at identifying true positives (precision), it misses a significant portion of relevant cases (recall). Regarding feature importance, both features, pos (0.4914) and neg (0.5086), have nearly equal influence on the model's predictions. The slightly higher importance of the neg feature suggests that negative sentiment might have a marginally stronger impact on the recommendation classification compared to positive sentiment. This balance reflects the fact that both positive and negative sentiment are essential in determining whether a review leads to a recommendation or not.

4. Redo Part 3 with a neural network model. Set one hidden layer with 30 neurons and use logistic as the activation function for the model. Report the recall and precision scores and obtain the SHAP summary plot (set the plot_type as violin) on the prediction. Comment on the results.

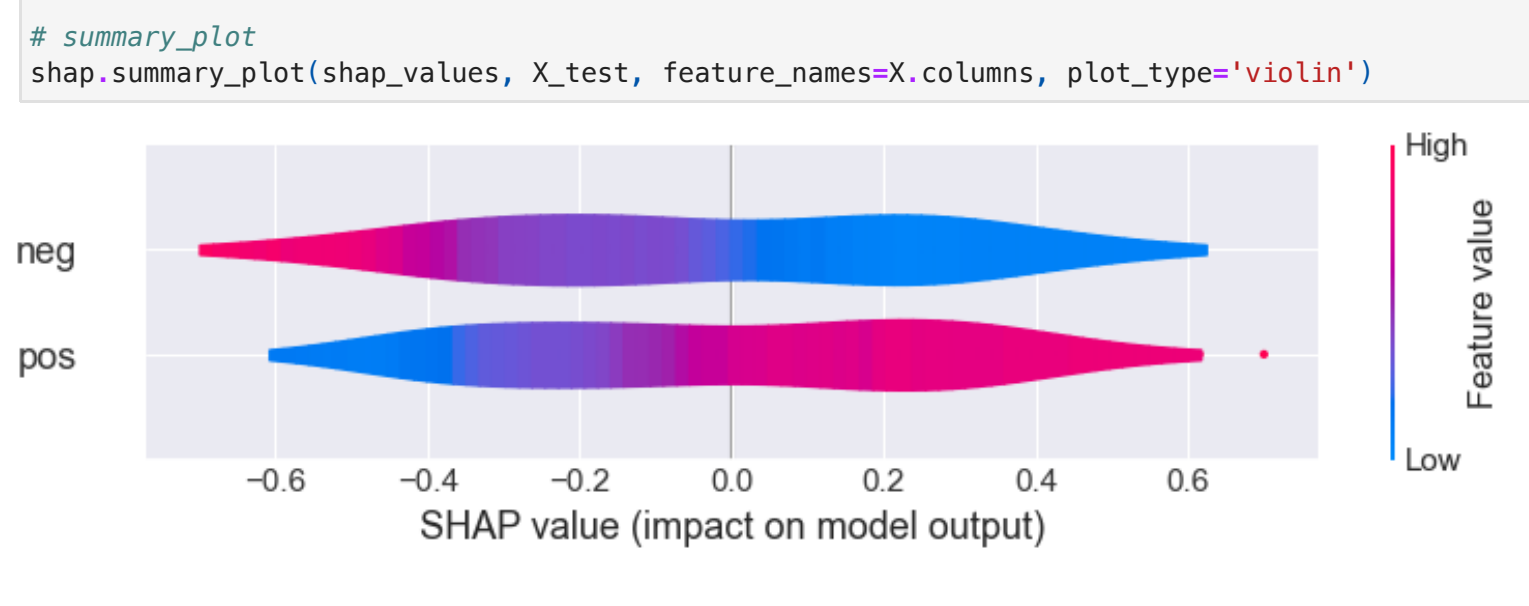
In [37]:
```
# hidden_layer_sizes=(): no hidden layer
# very much like a logistic regression if activation is 'logistic'
# "alpha=0" means no regularization
NN = MLPClassifier(random_state=1080, max_iter=1000, hidden_layer_sizes=30, activation='logistic', alpha=0)
pipe_NN = make_pipeline(StandardScaler(), NN)
pipe_NN.fit(X_train, y_train)
```

Out[37]:
```
▸ Pipeline
▸ StandardScaler
▸ MLPClassifier
```

In [38]:
```
# get predicted values
yp_NN = pipe_NN.predict(X_test)

# Calculation based on yp
print("Precision =", precision_score(y_test, yp_NN).round(3))
print("Recall =", recall_score(y_test, yp_NN).round(3))
```

```
Precision = 0.815
Recall = 0.707
```

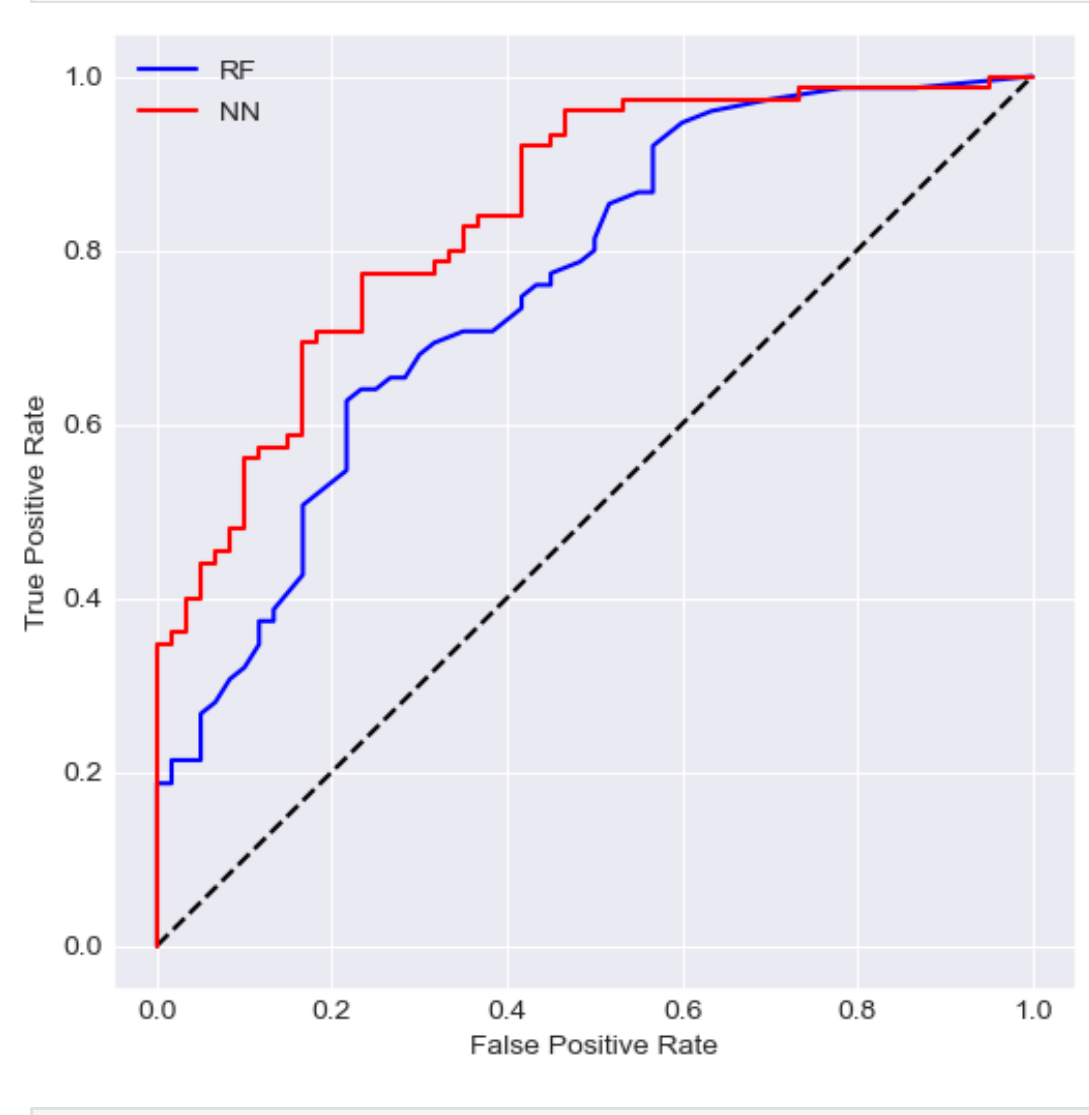
In [39]:
```
# shap Explainer
explainer = shap.Explainer(pipe_NN.predict, X_train.values)
shap_values = explainer(X_test.values)

# summary_plot
shap.summary_plot(shap_values, X_test, feature_names=X.columns, plot_type='violin')
```



The neural network with one hidden layer of 30 neurons and a logistic activation function resulted in a precision of 0.815 and a recall of 0.707. These metrics indicate that the model performs well, identifying true positives with a high degree of precision while slightly compromising recall, meaning it misses some relevant cases. The SHAP summary plot shows the feature contributions of pos and neg to the model's output. Both features have a substantial impact, with neg having a slightly stronger influence. The violin plot further illustrates the range and distribution of SHAP values, indicating how variations in the pos and neg features affect the model's prediction, with both high and low values of neg feature contributing significantly to the prediction.

5. Use the results of Parts 3 and 4 to construct ROC curves. Does neural network outperform random forest? Explain.

In [40]:
```
yp_p_RF = RF.predict_proba(X_test)
fpr_RF, tpr_RF, th_RF = roc_curve(y_test, yp_p_RF[:,1])

yp_p_NN = pipe_NN.predict_proba(X_test)
fpr_NN, tpr_NN, th_NN = roc_curve(y_test, yp_p_NN[:,1])

plt.figure(figsize=(6,6))
plt.plot([0,1], [0,1], color="black", linestyle="--")
plt.plot(fpr_RF, tpr_RF, color="blue", label="RF")
plt.plot(fpr_NN, tpr_NN, color="red", label="NN")
plt.ylabel("True Positive Rate")
plt.xlabel("False Positive Rate")
plt.legend()
plt.show()
```



In [41]:
```
# calculate AUC
print(f"Area Under ROC Curve for Decision Tree: {roc_auc_score(y_test, yp_p_RF[:,1])}")
print(f"Area Under ROC Curve for Neural Network: {roc_auc_score(y_test, yp_p_NN[:,1])}")
```

```
Area Under ROC Curve for Decision Tree: 0.758
Area Under ROC Curve for Neural Network: 0.8417777777777777
```

The ROC curves indeed show that the neural network (NN) outperforms the random forest (RF) based on the area under the curve (AUC). The neural network's curve (in red) stays closer to the top-left corner, indicating a higher true positive rate for a given false positive rate, compared to the random forest's curve (in blue). This suggests that the neural network has better overall predictive performance in this context, especially when distinguishing between positive and negative classes. While both models perform better than random guessing (the dashed black line), the neural network demonstrates superior classification ability based on the ROC analysis.

Question 3 (17.5 marks)

Use the data in `import_export.csv` to answer this question (load it as a dataframe called `MX`). The dataset contains two monthly series: international arrivals and departures of Australia.

1. Make line plots for the two series in the dataframe `MX`. Comment on the plot.
2. Define a subset of the dataframe ad between `2000-01-01` and `2019-12-01` as `MX20`. Make seasonal-trend-cycle decomposition of the two series in `MX20` and use the outcomes to compare the trend & seasonality of them. Set model to `additive`.
3. Transform `MX20` from monthly frequency to be quarterly (accumulated) and call it `MX20_q`. How many quarters are in the sample and for how many quarters are `Imports` more than `Exports`?
4. Use the entire set of `Imports` in `MX20_q` to forecast into the future imports of the next `2` years with an `AR(8)` model. Report & comment on the forecasting.
5. Now, split `MX20_q` into test and train sets by holding the last `8` quarters as the test and the rest as the train set. Train the following three models with `16` lags (i) triple exponential smoothing (set method="L-BFGS-B" with additive trend and seasonality) (ii) LASSO (set the penalty parameter as 1 & increase max_iter if necessary) (iii) Gradient Boosting on the train set, and apply the fitted model to predict the test set. Compare the performance of the three models.

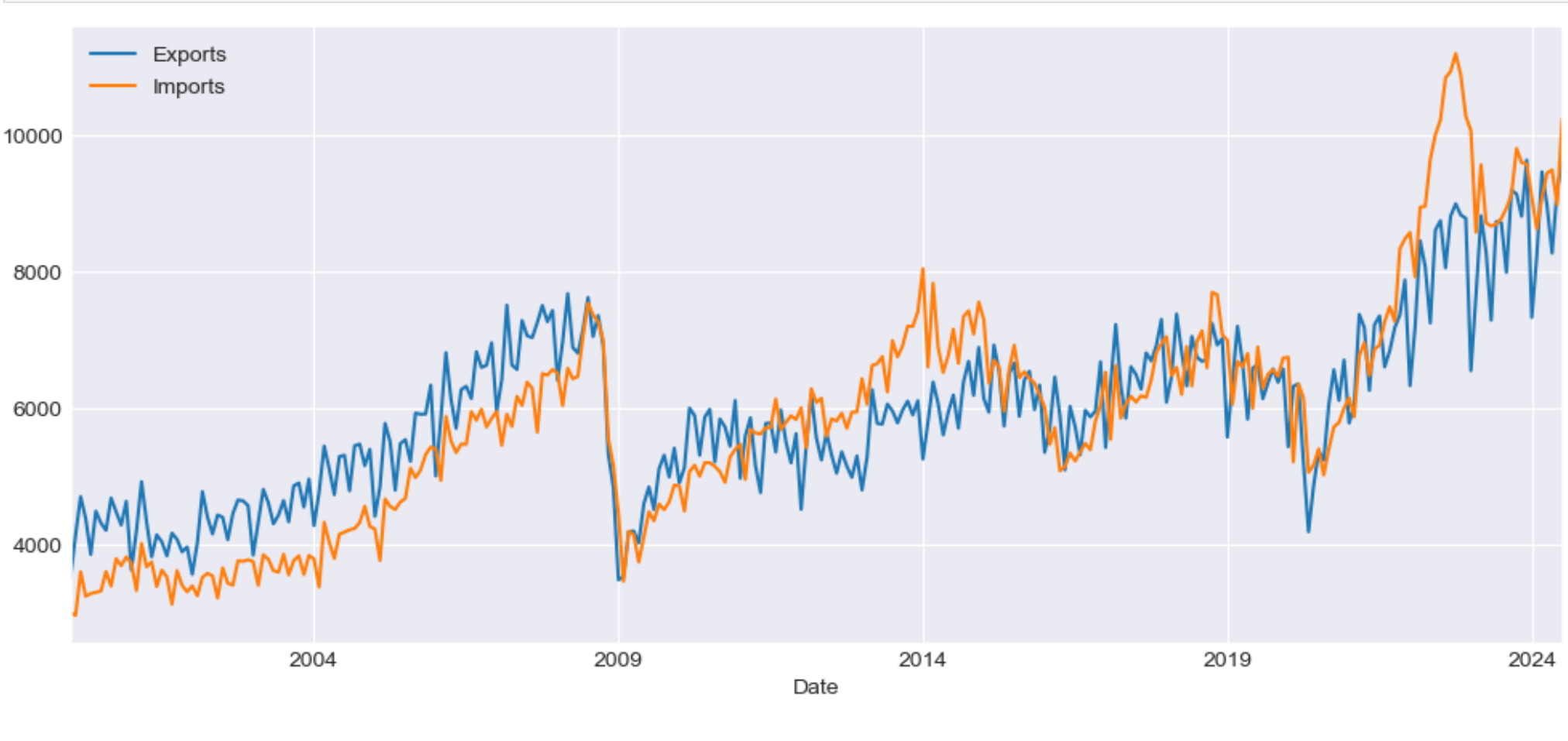
In [42]:
```
# load data
MX = pd.read_csv('import_export.csv')
# assign date
MX['Date'] = pd.to_datetime(MX['Date'], format='%Y/%m', errors='coerce')
# set Date as the index
MX.set_index('Date', inplace=True)
MX.head(3)
```

Out[42]:

	Exports	Imports
Date		
2000-01-01	3512.721254	2992.192350
2000-02-01	4136.816214	2957.534578
2000-03-01	4700.061682	3594.783576

1. Make line plots for the two series in the dataframe MX. Comment on the plot

In [43]:
```
MX.plot(figsize=(12,5))
plt.show()
```



The line plot shows the monthly trends of international arrivals (in blue) and departures (in orange) for Australia over time. Both series exhibit a clear upward trend, indicating increasing movement into and out of the country. There are noticeable fluctuations, with both series generally moving together, although departures (orange) consistently exceed arrivals (blue) in recent years. A significant dip is visible around 2009, likely reflecting a global or national event that temporarily reduced both arrivals and departures. Additionally, there appears to be a seasonal pattern in both series, with regular peaks and troughs throughout the years. Overall, the plot highlights the close relationship between the two series, with similar seasonal patterns and growth.

2. Define a subset of the dataframe ad between 2000-01-01 and 2019-12-01 as MX20. Make seasonal-trend-cycle decomposition of the two series in MX20 and use the outcomes to compare the trend & seasonality of them. Set model to additive

In [44]:
```
MX20 = MX['2000-01-01':'2019-12-01']
MX
```

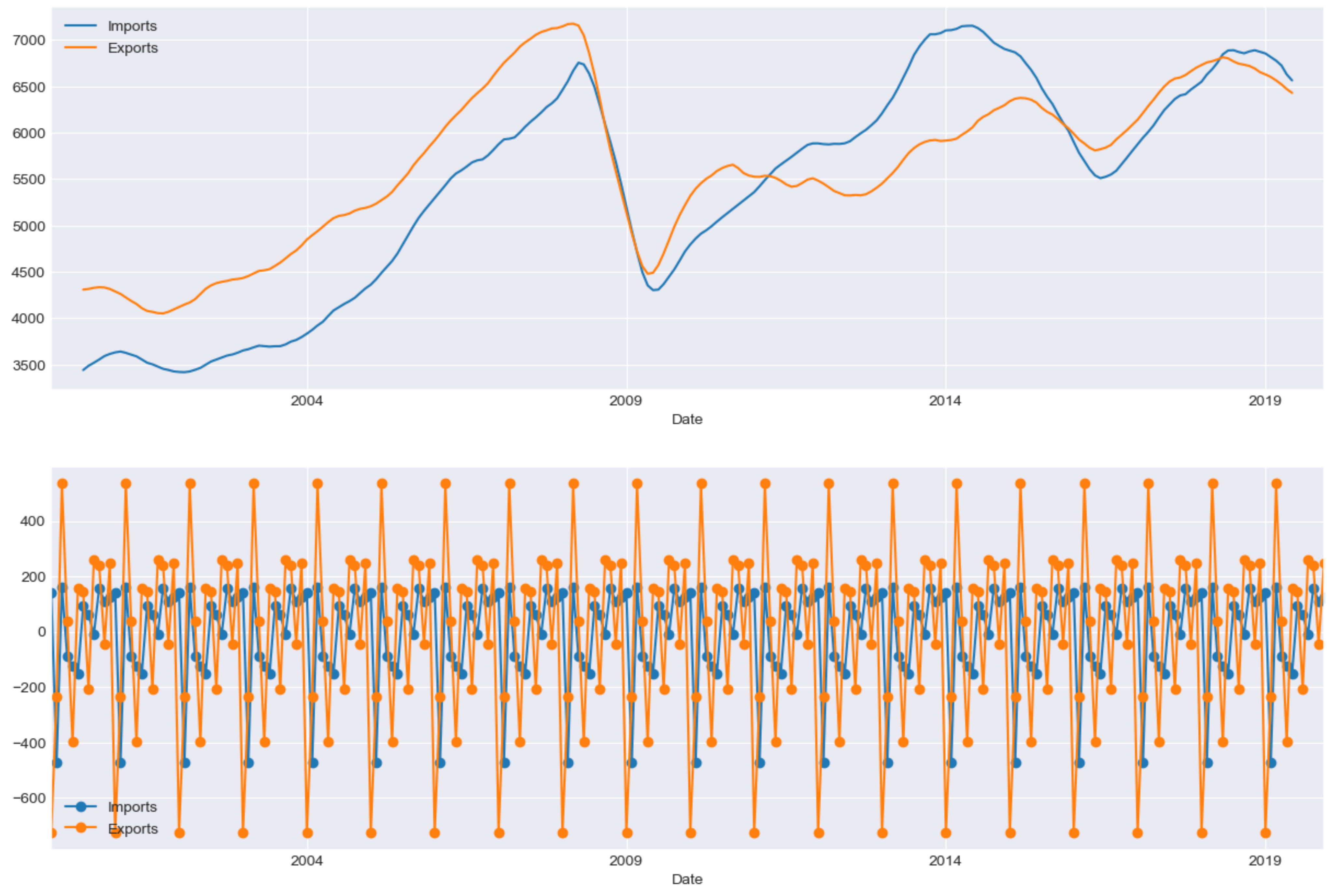
Out[44]:

Date	Exports	Imports
2000-01-01	3512.721254	2992.192350
2000-02-01	4136.816214	2957.534578
2000-03-01	4700.061682	3594.783576
2000-04-01	4377.801174	3236.988257
2000-05-01	3851.459857	3276.435493
...	...	...
2024-03-01	9469.324845	9086.925557
2024-04-01	8980.075715	9451.403119
2024-05-01	8276.854693	9499.896991
2024-06-01	9209.062240	8989.641078
2024-07-01	9612.707031	10246.964220

295 rows × 2 columns

In [45]:
```
imp = seasonal_decompose(MX20['Imports'], model='additive')
exp = seasonal_decompose(MX20['Exports'], model='additive')
```

In [46]:
```
# figsize
fig, (ax1,ax2) = plt.subplots(2,1, figsize=(15,10))

imp.trend.plot(ax=ax1, style='-', legend=True, label='Imports')
exp.trend.plot(ax=ax1, style='-', legend=True, label='Exports')
imp.seasonal.plot(ax=ax2, style='o-', legend=True, label='Imports')
exp.seasonal.plot(ax=ax2, style='o-', legend=True, label='Exports')
plt.show()
```



The seasonal-trend-cycle decomposition of the MX20 dataset shows clear trends and seasonality for both imports and exports between 2000 and 2019. The trend component (top plot) indicates that both imports and exports have experienced long-term growth, with exports generally outpacing imports during this period. Both series show a significant dip around 2009, likely due to the global financial crisis, followed by recovery and fluctuations. The seasonality component (bottom plot) reveals regular and consistent patterns in both series, with exports showing larger seasonal fluctuations than imports. This indicates that exports are more sensitive to seasonal factors, while imports exhibit a less pronounced but still regular seasonal pattern. Overall, the decomposition shows that while both series follow a similar long-term trend, their seasonal behaviors differ, with exports being more variable throughout the year.

3. Transform MX20 from monthly frequency to be quarterly (accumulated) and call it MX20_q. How many quarters are in the sample and for how many quarters are Imports more than Exports?

In [47]:
```
MX20_q = MX20.resample(rule='Q').sum()
```

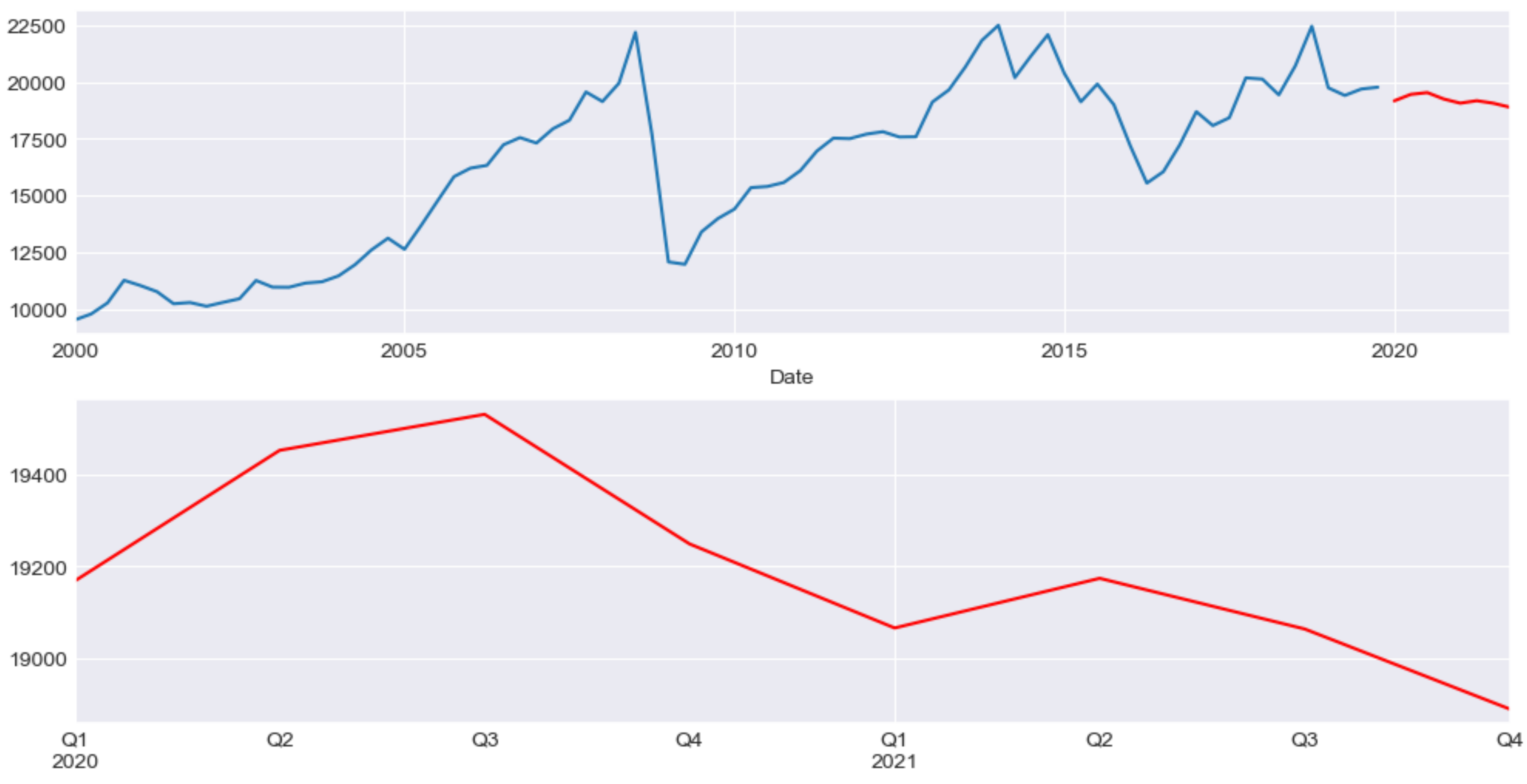
In [48]:
```
print(f"There are {len(MX20_q)} quarters in the sample")
```
```
There are 80 quarters in the sample
```

In [49]:
```
print(f"Number of quarters with more Imports than Exports: {sum(MX20_q['Imports'] > MX20_q['Exports'])} quarters")
```
```
Number of quarters with more Imports than Exports: 29 quarters
```

4. Use the entire set of Imports in MX20_q to forecast into the future imports of the next 2 years with an AR(8) model. Report & comment on the forecasting.

In [50]:
```
# Assign lags & steps
(n_lags, m_steps) = (8, 8) # 2 years

# Define model
model = LinearRegression()

# Apply ForecasterAutoreg
forecaster = ForecasterAutoreg(regressor=model, lags=n_lags)
forecaster.fit(y=MX20_q['Imports'])

# Make predictions
predictions = forecaster.predict(steps=m_steps)
```

In [51]:
```
# plot
fig, axs = plt.subplots(2,1, figsize=(12,6))

MX20_q['Imports'].plot(ax=axs[0])
predictions.plot(ax=axs[0], color='r')
predictions.plot(ax=axs[1], color='r');
```



The forecasted values for imports over the next 8 quarters (2 years) using the AR(8) model show a slight decline in the near future. The top plot shows the historical data for imports, with a steady increase over the years and occasional fluctuations. The forecast, shown in red in both plots, predicts a mild downward trend after the year 2020. The model indicates that imports will stabilize, but with a gradual decrease across the forecast period. This forecast seems reasonable given the recent flattening in the historical data, although it suggests a potential shift in the trend that warrants further investigation, especially if external factors like economic conditions are involved.

5. Now, split MX20_q into test and train sets by holding the last 8 quarters as the test and the rest as the train set. Train the following three models with 16 lags (i) triple exponential smoothing (set method="L-BFGS-B" with additive trend and seasonality) (ii) LASSO (set the penalty parameter as 1 & increase max_iter if necessary) (iii) Gradient Boosting on the train set, and apply the fitted model to predict the test set. Compare the performance of the three models.

In [52]:
```
m_steps = 8
idx = len(MX20_q) - m_steps

train = MX20_q[:idx]
test = MX20_q[idx:]
print(f"length of (train : test) = ({len(train)} : {len(test)}) ")
```
```
length of (train : test) = (72 : 8)
```

In [53]:
```
train_imp = train['Imports']
test_imp = test['Imports']

# Assign lags & steps
(n_lags, m_steps) = (16, 8)
```

(i) triple exponential smoothing

In [54]:
```
# fit train set
fitted_model = ExponentialSmoothing(train_imp, seasonal='add', trend='add').fit(method="L-BFGS-B")

# forecast test
pred_ES = fitted_model.forecast(m_steps)
```

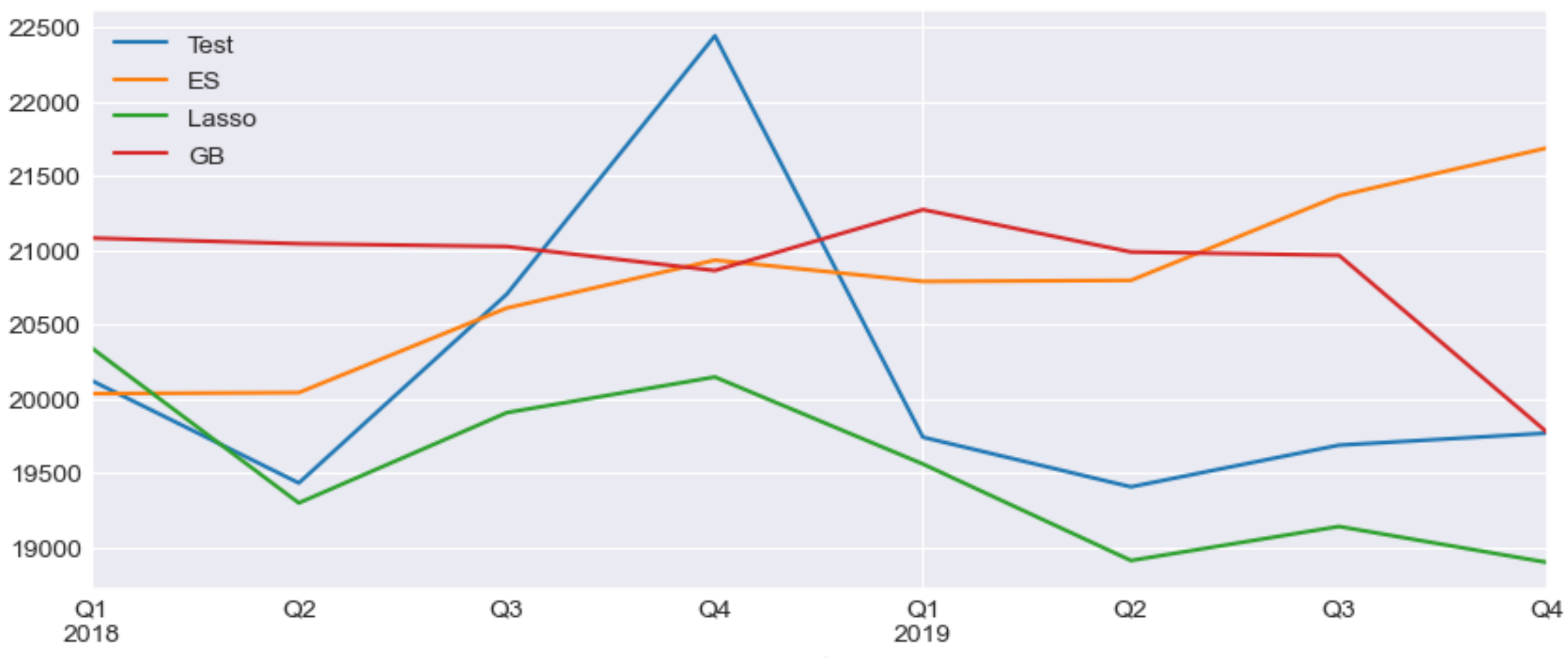
(ii) LASSO

In [55]:
```
# Define model
model = Lasso(alpha=1, max_iter=10000)
pipe = make_pipeline(StandardScaler(), model)

# Apply ForecasterAutoreg
forecaster_Lasso = ForecasterAutoreg(regressor=pipe, lags=n_lags)
forecaster_Lasso.fit(y=train_imp)

# Make predictions
pred_Lasso = forecaster_Lasso.predict(steps=m_steps)
```

(iii) Gradient Boosting

In [56]:
```
# Define model
GB = GradientBoostingRegressor(random_state=1080)

# Apply ForecasterAutoreg
forecaster_GB = ForecasterAutoreg(regressor=GB, lags=n_lags)
forecaster_GB.fit(y=train_imp)

# Make predictions
pred_GB = forecaster_GB.predict(steps=m_steps)
```

In [57]:
```
# test set only
test_imp.plot(legend=True, label='Test')
pred_ES.plot(legend=True, label='ES', figsize=(10,4))
pred_Lasso.plot(legend=True, label='Lasso', figsize=(10,4))
pred_GB.plot(legend=True, label='GB', figsize=(10,4))
plt.title('Predictions')
plt.show()
```



In [58]:
```
# Compute MSE
print(f'MSE (ES) = {mean_squared_error(test_imp, pred_ES).round(0)}')
print(f'MSE (Lasso) = {mean_squared_error(test_imp, pred_Lasso).round(0)}')
print(f'MSE (GB) = {mean_squared_error(test_imp, pred_GB).round(0)}')
```
```
MSE (ES) = 1526925.0
MSE (Lasso) = 915190.0
MSE (GB) = 1575430.0
```

The comparison of the three models—Triple Exponential Smoothing (ES), LASSO, and Gradient Boosting (GB) reveals notable differences in performance based on the mean squared error (MSE) values. LASSO achieves the lowest MSE (915190), indicating that it performs best at predicting the test set. Triple Exponential Smoothing (MSE: 1526925) and Gradient Boosting (MSE: 1575430) both have significantly higher error rates, suggesting that these models are less accurate in this case. The plot further demonstrates this: LASSO follows the trend of the test data more closely, while ES and GB are more stable but less responsive to fluctuations. In summary, LASSO provides the most accurate predictions of the three models, followed by ES and GB.

End of the Exam.